SMU Lyle School of Engineering

Reinforcement Learning

Temporal-Difference Learning

Sutton/Barto* Chapter 6

Michael Hahsler, SMU

With figures from Sutton/Barto*

*Sutton and Barto, Reinforcement Learning: An Introduction, 2nd edition, MIT Press, Cambridge, MA, 2018

Summary of Notation

General

X	capital letters: random variables
x, p	lower-case letters: realizations of random variables or scalar functions
$\boldsymbol{w}$	Bold lower-case letters: real-valued vectors (even if random variables)
$\boldsymbol{W}$	bold capitals: matrices
α	Greek letters: parameters (vectors if in bolt)
$\Pr\{X = x\}$	probability that a random variable X takes on the value x
$X \sim p$	random variable X selected from distribution $p(x) = \Pr\{X = x\}$
$\mathbb{E}[X]$	expectation of a random variable X, i.e., $\mathbb{E}[X] = \sum_x p(x)x$
$\text{argmax}_a f(a)$	a value of action a at which $f(a)$ takes its maximal value

MDP

s, s'	states
a	an action
r	a reward
$\mathcal{S}$	set of all (nonterminal) states, $\mathcal{S}^+$ are all states
$\mathcal{A}(s)$	set of all actions available in state s
γ	discount-rate parameter
t	discrete time step
T	final time step of an episode (a.k.a. horizon)
A_t	random variable for the action at time t
S_t	random variable for the state at time t
R_t	random variable for the reward at time t
$p(s', r \mid s, a)$	probability of transition to state s' and receiving reward r, from state s taking action a
	expected immediate reward from state s after action a
$\pi(a \mid s)$	probability of taking action a in state s under stochastic policy π
$\pi(s)$	action taken in state s under deterministic policy π

Value Function

G_t	return (cumulative reward) following time t
$G_{t:h}$	return from t to h (discounted and corrected)
$v_\pi(s)$	value of state s under policy π (expected return)
$v_*(s)$	value of state s under the optimal policy
V, V_t	array estimates of state-value function v_π or v_*
$q_\pi(s, a)$	value of taking action a in state s under policy π
$q_*(s, a)$	value of taking action a in state s under the optimal policy
Q, Q_t	array estimates of action-value function q_π or v_*

Temporal Differencing

U_t	target for estimate at time t
δ_t	TD error

Temporal-Difference (TD) Learning

- We will first focus on:
  - On-step
  - Tabular
  - Model-free TD

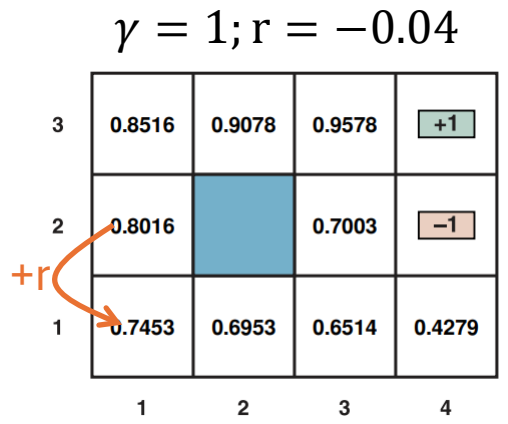
- **Idea**: Make the state value estimate $V(S_t)$ more similar to the immediate reward plus the discounted, estimated value of the next state $V(S_{t+1})$:

$$V(S_t) \leftarrow R_{t+1} + \gamma V(S_{t+1})$$

- The difference is called the **DT error**: $\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$

Value at different time steps

- Combines ideas from MC and DP
  - From MC: learn from raw experience (observed $S_t, A_t, R_{t+1}, S_{t+1}$)
  - From DP: calculates the error between two value estimates without waiting for the end of the episode via bootstrapping.



TD Prediction

Estimate the value function for a policy.

TD Prediction

- Follow π and update estimate V of v_π
- MC: $V(S_t) \leftarrow \mathrm{V}(\mathrm{S_t}) + \alpha[G_t - V(S_t)]$

MC Error over complete episode

Compare with the incremental update Implementation of MC methods.

- TD: $V(S_t) \leftarrow \mathrm{V}(\mathrm{S_t}) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$

TD Error for one step

Uses reward + estimate for next state = target U_t

- Updating an estimate using another estimate is called **“bootstrapping.”**

Alg. Tabular TD(0)

Tabular TD(0) for estimating v_π

Input: the policy π to be evaluated
Algorithm parameter: step size $\alpha \in (0, 1]$
Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(terminal) = 0$

Loop for each episode:
  Initialize S
  Loop for each step of episode:
    $A \leftarrow$ action given by π for S
    Take action A, observe R, S'
    $V(S) \leftarrow V(S) + \alpha\big[R + \gamma V(S') - V(S)\big]$
    $S \leftarrow S'$
  until S is terminal

Note: We can only update for time t at time $t + 1$ since we need to observe a complete S, A, R, S' interaction and S' is observed at $t + 1$.

Advantages of TD Prediction Methods

- **Online updates** at each step. Bootstraps using estimate $\mathrm{V}(S')$ for the value of the next state.
  MC has to wait for the end of the episode.

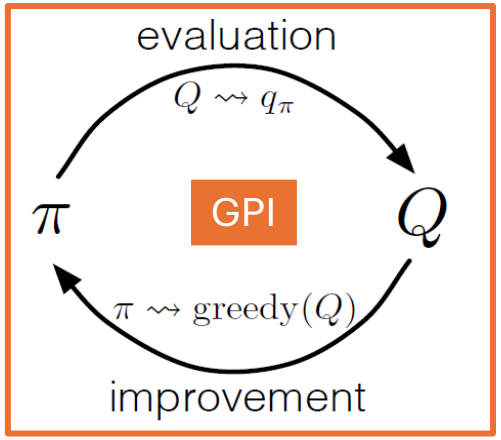
- **Model-free:** Only needs samples.
  DP needs the transition probabilities!

- **Convergence**: For any fixed π, TD(0) converges to v_π with probability 1 if α decreases sufficiently slowly over time.
  Converges typically faster than MC prediction.

Sarsa: On-policy TD Control

Find a good policy online while you use it.

Sarsa



- **Idea**: Use GPI with TD methods for policy evaluation.
- **Evaluation**: Estimate $q_\pi(s, a)$ using a Q-table with updates:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

$Q(S_{t+1}, A_{t+1})$ for a terminal state is defined as 0.

- **Improvement**: Create a soft policy based on Q.
- **Note**: Update uses $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$ which leads to the name **Sarsa.**

Alg. Sarsa

Tabular TD(0) for estimating v_π

Input: the policy π to be evaluated
Algorithm parameter: step size $\alpha \in (0, 1]$
Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(terminal) = 0$

Loop for each episode:
  Initialize S
  Loop for each s
    $A \leftarrow$ action gi
    Take action A, obs
    $V(S) \leftarrow V(S) + \alpha$
    $S \leftarrow S'$
  until S is terminal

The learned and followed policy is expressed as a Q-function.

Sarsa (on-policy TD control) for estimating $Q \approx q_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$
Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(terminal, \cdot) = 0$

Loop for each episode:
  Initialize S
  Choose A from S using policy derived from Q (e.g., ε-greedy)
  Loop for each step of episode:
    Take action A, observe R, S'
    Choose A' from S' using policy derived from Q (e.g., ε-greedy)
    $Q(S, A) \leftarrow Q(S, A) + \alpha\big[R + \gamma Q(S', A') - Q(S, A)\big]$
    $S \leftarrow S'$; $A \leftarrow A'$;
  until S is terminal

Follow the policy derived from the Q-function + update the Q-function.

Q-Learning: Off-policy TD Control

Find a good policy.

Q-Learning

- An extremely influential method developed by Watkins (1989).
- Update rule only uses observed Sars:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)]$$

The next action has not yet been observed. It assumes the greedy/optimal action will be chosen!

- **Off–policy**: Q directly approximates q_* independent of the behavior policy being followed!
- **Converges** to the optimal value function if:
  a) State-action pairs are represented discretely (in a table).
  b) All action are repeatedly sampled in all states (behavior policy keeps exploring).

Alg. Q-Learning

Sarsa (on-policy TD control) for estimating $Q \approx q_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$
Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(terminal, \cdot) = 0$

Loop for each episode:
  Initialize S
  Choose A from S
  Loop for each step
    Take action A, obse
    Choose A' from S'
    $Q(S, A) \leftarrow Q(S, A)$
    $S \leftarrow S'$; $A \leftarrow A'$;
  until S is terminal

Q-learning (off-policy TD control) for estimating $\pi \approx \pi_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$
Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(terminal, \cdot) = 0$

Loop for each episode:
  Initialize S
  Loop for each step of episode:
    Choose A from S using policy derived from Q (e.g., ε-greedy)
    Take action A, observe R, S'
    $Q(S, A) \leftarrow Q(S, A) + \alpha\left[R + \gamma \max_a Q(S', a) - Q(S, A)\right]$
    $S \leftarrow S'$
  until S is terminal

Does not use the actual A' for the update! Uses instead what Q thinks would be the optimal next action.

Expected Sarsa: Off-policy TD Control

Find a good policy.

Expected Sarsa

- Like Q-learning, but use the expectation over all possible next actions instead of the currently best action:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma\, \mathbb{E}_\pi[\, Q(S_{t+1}, A_{t+1}) |\, S_{t+1}] - Q(S_t, A_t)]$$

$$= Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \sum_a \pi(a|S_{t+1}) Q(S_{t+1}, a) - Q(S_t, A_t) \right]$$

- **Properties:**
  - Moves in the direction of Sarsa (in expectation).
  - Eliminates Sarsa's variance due to the selection of A_{t+1} from a soft policy.
  - Can set the learning rate $\alpha = 1$ since it uses expectation!
  - Can be used off-policy → generalizes Q-learning.

Maximization Bias

Q-Learning's Maximization Bias

- Q-learning update:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)]$$

- The next action is not observed, but the next best action according to Q is used in the update of Q.
- This cycle creates a **maximization bias** that leads to overestimating the Q-values.

- Solution:
  - **Double learning** by learning two independent Q-functions and using one for the action selection and the other one for the bootstrap estimate.

Double Q-Learning

- Learns two independent Q-functions. One is used for action selection and the other one for the bootstrap estimate.

Double Q-learning, for estimating $Q_1 \approx Q_2 \approx q_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$
Initialize $Q_1(s, a)$ and $Q_2(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, such that $Q(\mathit{terminal}, \cdot) = 0$

Loop for each episode:
  Initialize S
  Loop for each step of episode:
    Choose A from S using the policy ε-greedy in $Q_1 + Q_2$
    Take action A, observe R, S'
    With 0.5 probabilility:

$$Q_1(S, A) \leftarrow Q_1(S, A) + \alpha\Big(R + \gamma Q_2\big(S', \operatorname{argmax}_a Q_1(S', a)\big) - Q_1(S, A)\Big)$$

    else:

$$Q_2(S, A) \leftarrow Q_2(S, A) + \alpha\Big(R + \gamma Q_1\big(S', \operatorname{argmax}_a Q_2(S', a)\big) - Q_2(S, A)\Big)$$

    $S \leftarrow S'$
  until S is terminal

Bootstrap estimate

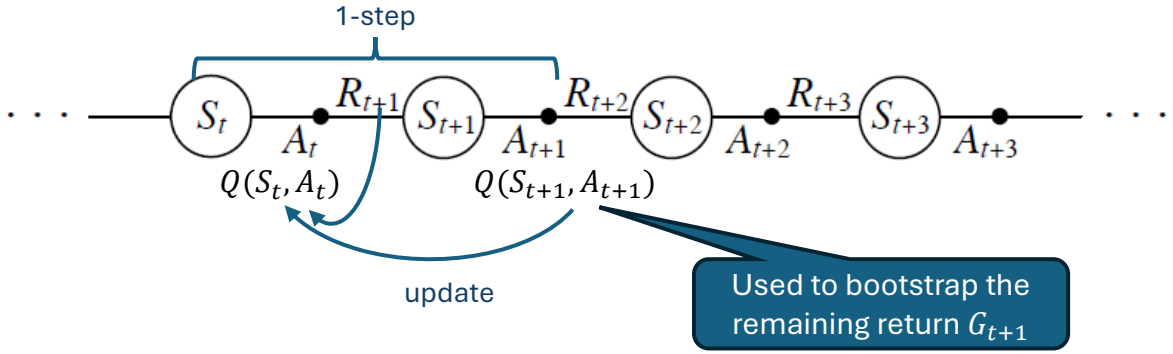
Action selection

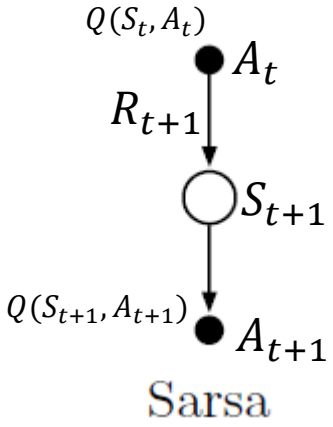
- Sarsa and Expected Sarsa have a similar bias and double learning extension for these algorithms also exist,

Comparison

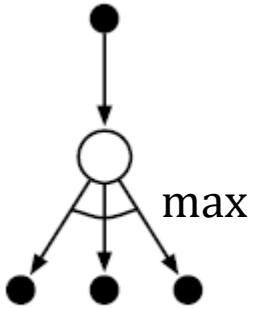
Sarsa vs. Q-Learning vs. Expected Sarsa

Comparison: 1-step Backup Diagrams

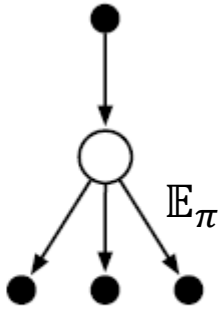




Sarsa



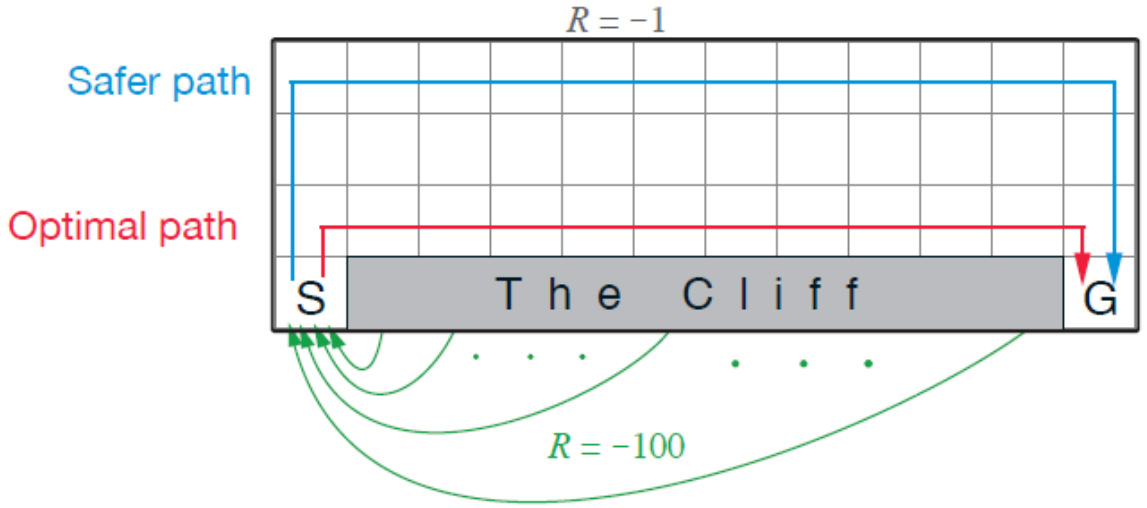
Q-learning



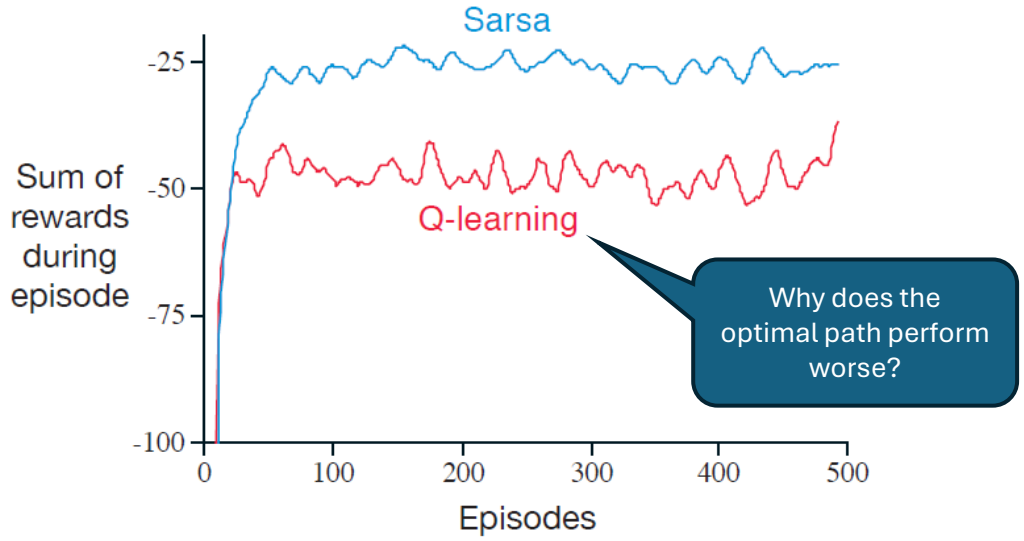
Expected Sarsa

Sarsa vs. Q-Learning

- **Cliff Walking**: Consider the gridworld shown to the right.
- This is a standard undiscounted, episodic task, with start and goal states.
- Actions causing movement up, down, right, and left.
- Reward is −1 on all transitions except those into the region marked “The Cliff.” Stepping into this region incurs a reward of −100 and sends the sends the agent instantly back to the start.



Performance of the Sarsa and Q-learning methods with ϵ-greedy action selection, $\epsilon = 0.1$.



Experimental Comparison

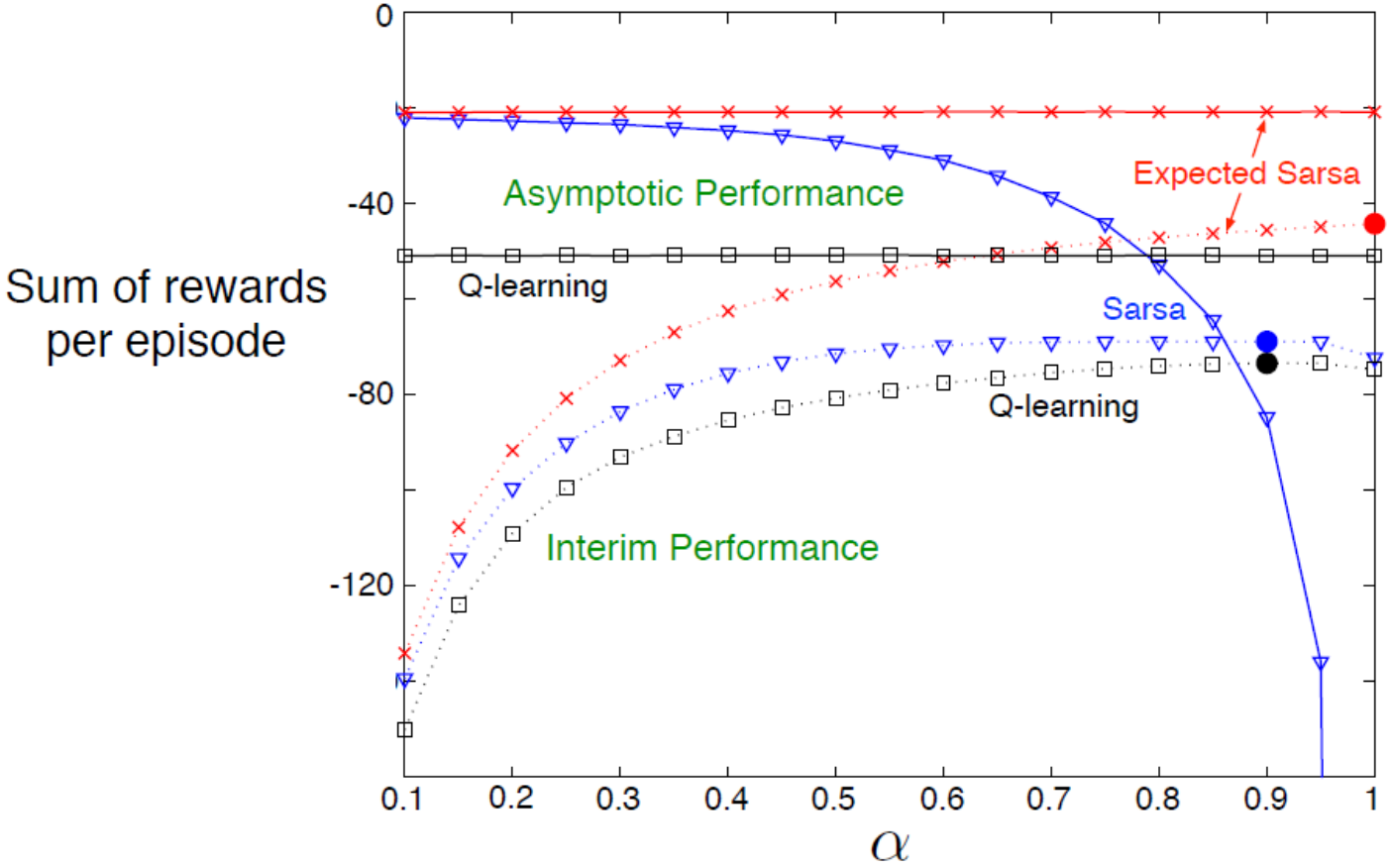


Figure 6.3: Interim and asymptotic performance of TD control methods on the cliff-walking task as a function of α. All algorithms used an ε-greedy policy with $\varepsilon = 0.1$. Asymptotic performance is an average over 100,000 episodes whereas interim performance is an average over the first 100 episodes. These data are averages of over 50,000 and 10 runs for the interim and asymptotic cases respectively. The solid circles mark the best interim performance of each method. Adapted from van Seijen et al. (2009).

What You Should Know

- Time difference (TD) learning is an alternative to MC for the prediction (policy evaluation) problem.

- **Method**: Stepwise reduces the TD error for the next step.

- Popular and most widely used RL methods:
  - On-policy: Sarsa
  - Off-policy: Q-learning
- Advantages:
  - Model-free: Uses experience.
  - Learn online (after each observation).
  - Simple with minimal computation.
- Challenges:
  - Maximation bias.
  - Behavior policy needs to keep exploring.
  - Not very sample efficient.

- We will extend the one-step tabular TD ideas from this chapter to $\boldsymbol{n}$**-steps** and **function approximation**.